WoSM Academy

CS102-F26 · DIGITAL DESIGN AND COMPUTER ARCHITECTURE I

Arithmetic

Lecture 16: half and full adders, multi-bit adders, and two's complement

Dr. Ali Khudiyev

Fall 2026

CC BY-SA 4.0

Today

01 Adding bits

02 Multi-bit adders

03 Negative numbers

04 Summary

01

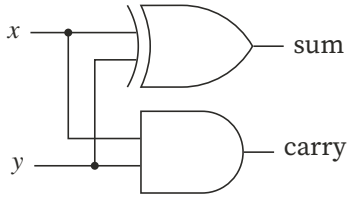
Adding bits

Adding two bits

x	y	sum	carry	$x + y$
0	0	0	0	00
0	1	1	0	01
1	0	1	0	01
1	1	0	1	10

Half adder (Table 6.3)

- $1 + 1 = 2 = 10_2$: the sum needs a second bit, the *carry*.
- sum $= x \oplus y$, carry $= x\,y$.



Why half is not enough

carries	1 1 0
x	0 1 1 0
$+\ y$	0 1 1 1
	1 1 0 1

$6 + 7 = 13$, by hand

- Adding numbers by hand, column by column from the right, each column adds three bits: x_i, y_i and the carry from the column on its right.
- A half adder has no input for that carry, so a row of half adders cannot add multi-bit numbers.
- We need an adder of three bits: the *full adder*.

The full adder

c_i	x_i	y_i	s_i	c_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

`Full adder (Table 6.4):`

$$c_i + x_i + y_i = [c_{i+1}\, s_i]$$

- The sum bit is 1 when an odd number of inputs are 1:

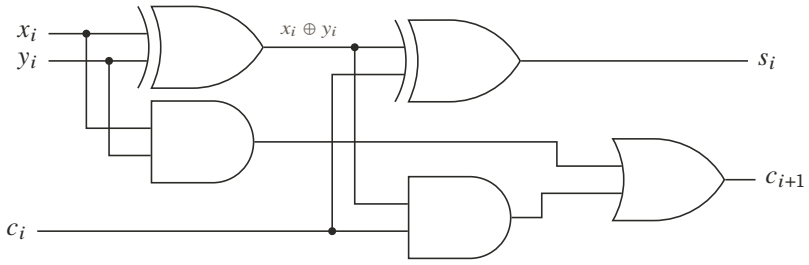
$$s_i = x_i \oplus y_i \oplus c_i.$$

- The carry is 1 when at least two inputs are 1:

$$c_{i+1} = x_i\, y_i + (x_i \oplus y_i)\, c_i = x_i y_i + x_i c_i + y_i c_i.$$

- Check the second form with a K-map `(Unit 5)`; the first is the book's circuit, on the next slide.

Two half adders and an OR

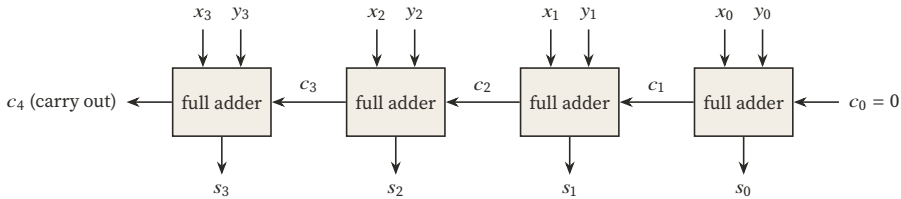


- The first half adder adds x_i and y_i; the second adds the carry c_i to their sum. A carry from either one makes $c_{i+1} = 1$.
- Five gates: two XOR, two AND, one OR.

02

Multi-bit adders

The ripple-carry adder



- One full adder per bit; each passes its carry to the next one on the left. The first carry in is 0. This is the book's 4-bit adder.
- The DX8 adds bytes, so it chains 8 full adders: an 8-bit adder.
- The carry *ripples* from bit 0 to bit $n - 1$, as the clock edge did through the ripple counter `(Unit 13)`.

The adder at work

carries	0 1 1 0 0	
x	0 1 1 0	
$+\ y$	0 1 1 1	
	0 1 1 0 1	

6 + 7 = 13: carry out $c_4 = 0$

carries	1 1 1 0 0	
x	1 0 1 1	
$+\ y$	0 1 1 0	
	1 0 0 0 1	

11 + 6 = 17: carry out $c_4 = 1$

- The carries row is $c_4 c_3 c_2 c_1 c_0$, with $c_0 = 0$.
- 4 bits hold 0 to 15. 17 does not fit: the adder gives 0001 and $c_4 = 1$. For unsigned numbers, a carry out means the result is too large.

How long does it take?

- s_{n-1} is correct only once the carry has come all the way from bit 0: the longest path goes through every full adder.
- So the propagation delay `(Unit 9)` of a ripple-carry adder grows with n: an 8-bit adder is about twice as slow as a 4-bit one.
- In a clocked circuit, that delay is part of t_{pd} in the setup constraint `(Unit 14)`.

> **Remark**
>
> Faster adders work out the carries in parallel (*carry-lookahead*), at the cost of more gates. For the DX8, the ripple-carry adder is enough.

03

Negative numbers

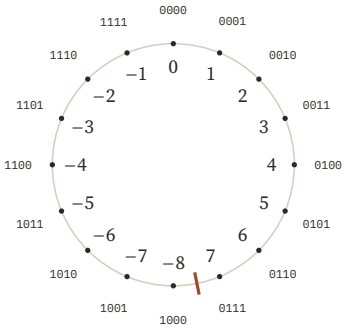
Two’s complement

Definition (Two’s complement)

In n-bit *two’s complement*, the bits $b_{n-1} \ldots b_1 b_0$ stand for

$$-b_{n-1}\,2^{n-1} + b_{n-2}\,2^{n-2} + \cdots + b_1\,2 + b_0.$$

- The leftmost bit counts -2^{n-1} instead of $+2^{n-1}$: it is 1 for every negative number.
- n bits hold -2^{n-1} to $2^{n-1} - 1$: -8 to 7 in 4 bits, -128 to 127 in a byte.



4-bit two’s complement

Negating: NOT a + 1

- a + NOT a = 11 . . . 1, which is −1 in two's complement. So

$$-a = \text{NOT } a + 1.$$

- Example, 4 bits: 5 = 0101, NOT 5 = 1010, plus 1: 1011 = −8 + 2 + 1 = −5.
- And back: NOT 1011 + 1 = 0100 + 1 = 0101 = 5.
- The exception: −8 = 1000 gives 0111 + 1 = 1000 again. There is no +8 in 4 bits.

> **The DX8's neg**
>
> `regOUT` ← NOT `regA` + 1 `(Unit 15)`. The book builds it from a second 8-bit adder: x = NOT a, y = 0, and carry in c_0 = 1.

One adder for both signs

- Two's complement numbers add with the same adder as unsigned ones: only the reading of the bits changes.
- 1101 + 1110: unsigned, 13 + 14 = 27, too large for 4 bits. In two's complement, −3 + (−2) = −5: the adder gives 1011, which is −5. The carry out is simply dropped.
- Subtraction for free: $a - b = a + \text{NOT}\ b + 1$. Example: 6 − 3 = 0110 + 1100 + 1 = 0011, dropping the carry out.

> **Remark**
>
> The DX8 has no `sub` instruction: `neg` and then `add` do the job.

Overflow

- In two's complement, a result outside -2^{n-1} to $2^{n-1} - 1$ comes out with the wrong sign:
  $5 + 4 = 0101 + 0100 = 1001 = -7$.
- It happens exactly when the carry into the leftmost bit differs from the carry out of it:

$$\text{overflow} = c_{n-1} \oplus c_n.$$

- That is the XOR gate on the *overflow* output of the book's 4-bit adder.

Sum	c_3	c_4	overflow
$5 + 4$	1	0	1
$-3 + (-2)$	1	1	0
$-8 + (-1)$	0	1	1
$-5 + 6$	1	1	0

Carry out (c_4): too large as unsigned.
Overflow: wrong as two's complement.

Try it

1. Add 1001 + 0111 with a 4-bit adder. What are the sum and the carry out?
   0000, carry out 1: 9 + 7 = 16.
2. Write −6 as an 8-bit two's complement number.
   6 = 0000 0110, NOT 6 + 1 = 1111 1010.
3. Which number is 1110 0100 in 8-bit two's complement?
   −28: NOT of it, plus 1, is 0001 1100 = 28.
4. Does 0110 + 0011 overflow in 4-bit two's complement?
   Yes: 6 + 3 = 9 > 7. The adder gives 1001 = −7; $c_3 = 1$, $c_4 = 0$.
5. How many gates does the DX8's 8-bit adder need?
   8 full adders of 5 gates each: 40 gates.

04

Summary

Summary

- Half adder: sum = $x \oplus y$, carry = xy. Full adder: it also adds a carry in; two half adders and an OR.
- A ripple-carry adder chains n full adders; its delay grows with n. A carry out means the unsigned result does not fit.
- Two's complement: the leftmost bit counts -2^{n-1}; n bits hold -2^{n-1} to $2^{n-1} - 1$.
- $-a = \text{NOT}\, a + 1$, and $a - b = a + \text{NOT}\, b + 1$: the same adder serves both signs.
- Overflow = $c_{n-1} \oplus c_n$: the two's complement result has the wrong sign.

Before the next lecture

- Read Section 6.2 of the book.
- In Logisim-evolution, build the half adder, the full adder, and a 4-bit ripple-carry adder with carry out and overflow. Test it on the sums of this lecture.
- Extend it to 8 bits: that is the DX8's adder.

> **Next lecture** (Unit 17)
>
> Logic and comparison: bitwise operations and comparators.